BIRMINGHAM CITY University

CMP6200/DIG6200

Individual Undergraduate Project (FYP)

Project Interim Report

Reinforcement Learning to improve AI behaviour and adaptability in a game environment.

Student Name: Tamer Hussen

Student ID: S23130437

Course: Computer Games Technology

Supervisor: Jan Krasniewicz

Date: 3/11/25

Abstract

The project investigates the use of reinforcement learning (RL) to produce adaptive, believable non player characters within a Unity game environment. Using Unity ML-Agents, the work will design controlled training environments, implement RL agents, and compare their behaviour against scripted baselines. Evaluation will combine quantitative training metrics with planned subjective measures to assess recognised responsiveness and believability. Outcomes aim to show practical trade-offs between designer control and emergent adaptability, and to produce reproducible guidance for using RL in playable game contexts.

List of Contents

List of Figures

List of Tables

1.0 Introduction and Context

The project investigates how Reinforcement Learning (RL) can make artificial intelligence (AI) agents more adaptive and capable in a video game context. Reinforcement Learning is an area of machine learning in which systems learn from experience, rewarding them for advantageous behaviours and punishing them for disadvantageous behaviours, rather than relying on prescribed rules (Sutton & Barto, 2018).

In modern game development, most AI or NPC systems use some form of pre-scripting or logic based on rules, which can lead to the AI agent becoming more predictable once the player observes the patterns built into the system (Isbister & Nass, 2000). By using RL, an AI agent can learn and improve while interacting with an environment, resulting in more engaging, realistic, and replayable experiences for the player (Mnih, et al., 2015) (Silver, et al., 2017).

The project will use Unity's ML-Agents toolkit to achieve machine learning based AI design (Juliani, et al., 2018). ML-Agents enables reinforcement learning directly within Unity using C#, allowing agents to interact with the game environment, receive feedback and adapt their behaviour through repeated simulations. This makes it possible to visualise and assess learning outcomes in real time, providing a practical and efficient way to develop intelligent, reactive game agents without relying on external Python integrations.

The main goal of the project is to develop and promote RL-based systems to those interested in bridging the gap between RL-focused academic research and practical applications of game AI, demonstrating that RL is much more advanced as a means of achieving behavioural complexity and realism in games (Senanayake, 2025). This practice is a huge area of commitment in modern games, where the adaptive capabilities of AI agents are of increasing importance for maintaining competition, experiencing immersion, and creating challenges for players.

1.1 Problem Statement

Normally, AI systems for games are fixed and predictable, basing their behaviours upon pre-defined scripts or decision trees that are unable to learn and adapt to the changing behaviour of the player (Zook, et al., 2019). The lack of adaptable AI limits player engagement and replayability.

The project responds to this issue by examining how Reinforcement Learning can promote a more adaptive, self-learning AI that can change its behaviours through the feedback that it derives from its environment, within Unity. The project aims to develop an AI that is able to learn and evolve while the game is being played, instead of relying on rigid, pre-defined logics.

2.0 Review of Existing Knowledge

Reinforcement Learning consists of various algorithmic families that differ in how agents learn and generalise from experience. The project focuses on comparing two or more representative RL methods, for example, proximal policy Optimization (PPO) and Deep Q-Networks (DQN), to evaluate which achieves more stable and believable learning within Unity ML-Agents.

The project specifically compares PPO and DQN methods implemented with in Unity ML-Agents to evaluate which algorithm produces more consistent and adaptable behaviour. While PPO is typically used for continuous control problems, DQN is suited for discrete actions, making them ideal for comparative study within a unified test environment.

Reinforcement Learning (RL) is based on the interaction between three main components: the agent, the environment, and the reward system. An Agent (AI character) takes actions, the environment gives feedback on those actions, and rewards help guide future decisions. Over time, the agent learns to maximise positive benefits through trial and error.

The main algorithms in RL include Q-Learning, Deep Q-networks (DQN), and Policy Gradient Methods, each providing different approaches to how agents learn from experience (Van Hasselt, et al., 2016) (Hester, et al., 2018).

Selected subtopics in reinforcement learning are particularly relevant for game AI and will frame the literature review and experiments: Policy optimisation methods, stable, sample-efficient methods commonly used in continuous control; Value-based methods and DQN variants, useful for discrete action tasks and simpler decision problems; Reward shaping and curriculum learning, techniques that influence learning speed, prevent reward hacking, and gradually increase task complexity; Transfer learning and domain adaptation, approaches to reuse learning policies across levels or maps, reducing retraining time; Multi-agent and

population-based training, methods that create robust behaviours through diverse opponents or co-learned populations.

Unity's ML-Agents toolkit provides a framework developed to integrate machine learning models within game environments (Juliani, et al., 2018). This allows developers to implement a simulated training session directly within Unity, often for use in experimentation in navigation, pathfinding, and decision-making based tasks, where agents continuously improve over time through repeated trials.

Previous research has shown some potential for RL in applied fields, including robotics, autonomous vehicles, and simulation- based control, but research into its applications in interactive video games is minimal (Pan & Yang, 2010) (Tang, 2023). Most of the RL work for video games falls under either experimental or non-playable simulation, meaning players are unable to see the benefits of the AI being capable of adaptive learning in real time and with a human player.

Although traditional game AI techniques (like finite state machines or behaviour trees) are simple and effective solutions, they lack adaptability (Bontrager, et al., 2018). Reinforcement learning (RL) enables perpetual learning which may result in AI agents that appear variously each time a player engages a game AI.

The project will compare different reinforcement learning algorithms, specifically PPO and DQN, to identify which produces more consistent, adaptive NPC behaviour. The evaluation will consider both training performance metrics and perceived realism when compared to traditional scripted AI.

This review establishes that although the potential of RL is acknowledged, its practical use in commercial or playable game AI is lacking, which is the core of the project.

2.1 Critical Analysis and Gap Identification

Although Reinforcement learning theory is studied and reviewed to a large extent, most of the studies tend to apply RL to controlled simulations rather than applying RL in gameplay environments. These studies fail to explore how learning affects player engagement or how it affects game balance or AI personality.

Most examples of training ML-Agents to address these issues in a Unity compatible format focus primarily on driving completion of a task (Senanayake, 2025). For example, getting an agent to move towards a goal, without analysing if learning has an impact on in-game adaptability or responsiveness (Anon, 2018). Research on

how reward structuring, environmental complexity, or sensory information affect the learning curve is also limited.

The identified key gaps are:

- There is a lack of studies applying RL in playable game contexts.
- Performance analysis for trained agents is uncommon beyond, success rate.
- There is little research exploring learned-agent performance metrics from reward design and environmental design perspectives.
- There is no research evaluating player perception of RL-based behaviour.

These gaps suggest the need for applied projects that demonstrate and measure how RL can make game AI smarter, adaptive, and more engaging.

2.2 Project Justification

The goal of the project is to address the previously identified gap by building a Unity-based environment designed to display reinforcement learning in a game-like setting. The AI agent would learn to complete certain tasks, such as navigation, avoidance, or collection, with appropriate adjustments to its behaviour depending on reward and penalty structures.

The project will also investigate how changing one or more aspects of the reward systems and complexity of the environment affect learning speed and behaviour in AI agents. In addition, the outcomes will compare the AI agent that was trained using RL with traditional scripted AI, showcasing the differences in learning and adaptability dynamics (Jaderberg, et al., 2019).

In addition, a questionnaire will capture feedback on how users perceive AI intelligence or realism, offering both technical and player-based feedback (Isbister & Nass, 2000).

The outcomes will help contribute to a better understanding of how reinforcement learning can be realised in or around real-time development practices, leading to increasingly immersive, evolving and replayable AI experiences.

Practical game examples and applied research will anchor the project. Candidate case studies include Unity ML-Agents demos, OpenAI /DeepMind multi-agent research showing complex emergent strategies, and published game – focused studies (Bontrager, et al., 2018) on automated playtesting and parameter tuning. The conference paper “Dynamic NPC AI Using reinforcement learning”

(Senanayake, 2025) specifically targets NPC behaviour in games and will be used as a comparative example. Each case study will be summarised for: problem addressed, RL approach, evaluation metrics used, and gaps that the project will attempt to cover.

3.0 Project Aims, Objectives, and Scope

3.1 Project Aim

The goal is to design, create and assess a reinforcement learning system in Unity that enhances AI's adaptability and decision-making processes when operating in a game environment.

3.2 Project Objectives

- **Objective 1:** investigate the principles of reinforcement learning (RL) and determine appropriate algorithms to run in the Unity ML-Agents framework
- **Objective 2:** Apply, assess, and train a reinforcement learning agent using a reward system to simulate adaptive behaviours.
- **Objective 3:** Create a Unity-based environment to mimic and monitor AI learning behaviours (like pathfinding or obstacle avoidance).
- **Objective 4:** Capture, visualise and analyse the agent's learning rate and behaviours across training sessions.
- **Objective 5:** Compare the performance and outcomes of the RL agent to a regular script-based AI and gather user feedback through questionnaires.
- **Objective 6:** Improve the reward structure and environment design to improve learning efficiency and realism.

3.3 Project Scope

Included:

- Incorporate the Unity ML-Agents toolkit to integrate reinforcement learning.
- Create a small-scale simulated environment.
- Implement adaptive AI behaviours and report on their characteristics.
- Visualisation of training progress (graphs, wireframes, or pathfinding visual examples).
- Collect basic feedback from users through the use of questionnaires.

Excluded:

- Multiplayer or online networked systems.
- Physical robotics or real-world machine learning applications.
- Deep learning architectures requiring high-end computing power.
- High-fidelity graphics or physics systems (focus is on AI behaviour, not visuals).

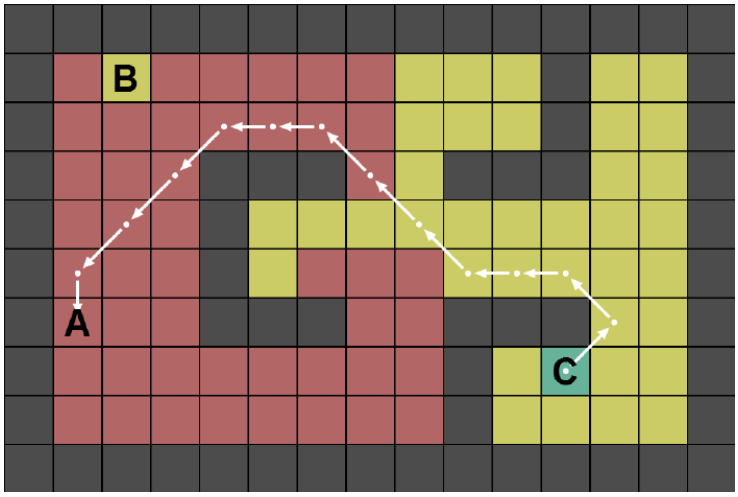
4.0 Project Design

4.1 Design and Methods

The project will follow a development and experimental design combination, which aligns software development with performance assessment. AI agents will be trained using Unity's ML-Agents toolkit, in a controlled environment simulating navigation or object interaction. Training will define rewards, penalties and environmental rules that will guide the learning process. The project will also collect and visualise data including cumulative rewards, completion rates, and learning curves. Moreover, feedback about learner perceptions will be collected through a small questionnaire that will assess how realistic or adaptive the AI feels compared to scripted options.

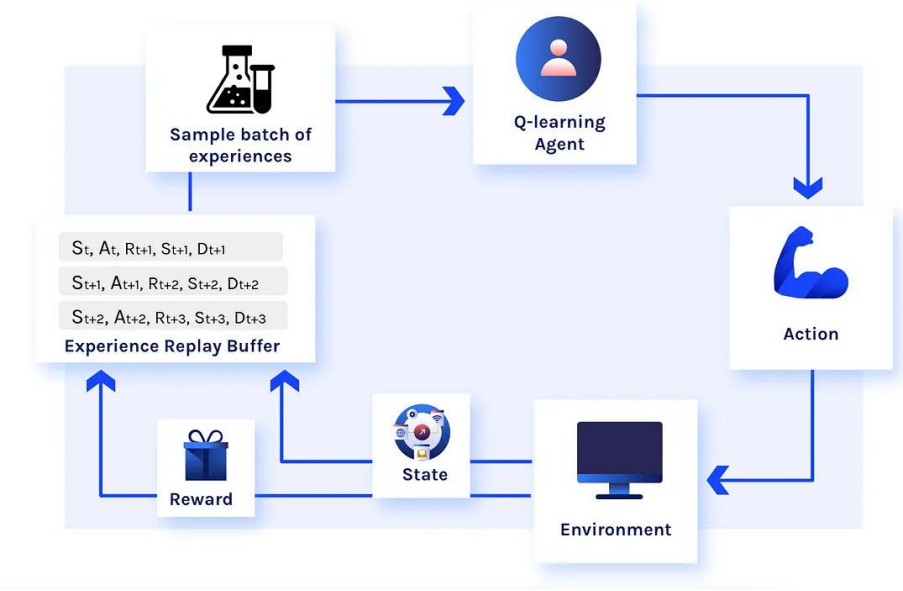
The project will use an Agile-inspired iterative development process, whereby progress will be reviewed weekly in short sprints. This allows for rapid testing and continuous refinement of the training setup. Methods such as scrum-style task planning and progress reviews will help manage scope, ensuring that training experiments remain achievable within the time limit.

Figure 1: Pathfinding



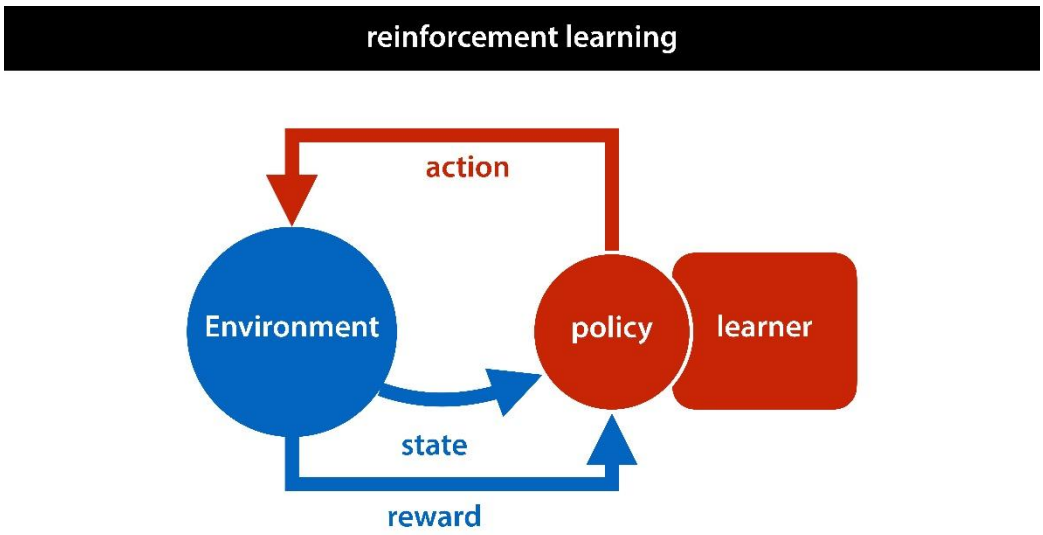
This figure shows C uses pathfinding to avoid obstacles and reach A, while B is represented as an intermediate waypoint in the route. (Khan, 2014)

Figure 2: RL agent pipeline



This figure shows how a reinforcement learning framework for a Q-learning agent works (with S representing current state, A being current action, R being reward, s being next state and D being the done signal). (Datatonic, 2022)

Figure 3: Reward shaping schematic



This figure shows how rewards can reinforce the agent to follow the policy and interact with the environment. (MLVU, 2022)

Planned evaluation

This is an interim plan for how the agent will be evaluated during the testing phase. Player testing has not yet been performed; the following tools are prepared for later data collection:

- Quantitative metrics: episodic cumulative reward, success rate, collisions, time to goal, path length.
- Planned questionnaire: a short Likert questionnaire to measure perceived responsiveness, believability and enjoyment. This questionnaire is included as planned instruments and will be used only during the testing phase when participants are available.
- Quantitative notes: brief session comments and observation logs. Data will be reported as averages with variance and compared across reinforcement learning and scripted baselines.

The questionnaire is a planned instrument for the testing phase, no participant data has been collected for this interim report. The full draft questionnaire is in Appendix A.

Planned Metrics

Primary metrics: cumulative episodic reward, success rate, collisions, and average time to goal. Secondary metrics: subjective user scores (Likert) and qualitative observations.

Comparison methodology

Each trained RL will be evaluated against a scripted baseline through identical scenarios trials to compare objective metrics, and side by side play sessions where observers, rate believability and responsiveness. Statistical tests will be used to confirm differences in metrics; effect sizes and confidence intervals will be reported. Qualitative behaviours will be logged and discussed.

4.2 Justification of Methods

Unity ML-Agents was selected due to its convenient basis for applying reinforcement learning within a game engine, this means there is not a need for an external simulations engine or interface. It allows for real time visualisation of the AI's experience, which can help analyse the learning process, and also to demonstrate the learning process to others.

Other machine learning frameworks, like TensorFlow or PyTorch alone, are less applicable to interactive environments and would require developing a custom process for visualisation (Lample & Chaplot, 2017). In contrast, Unity ML-Agents supports the simulation of environments, data collecting and monitoring of the agent behaviour in a single workflow, making it ideal for the project's scope.

The combination of technical development and player testing provides both quantitative and qualitative evaluation on how reinforcement learning influences AI behaviour in games.

4.3 Project Timeline

Figure 4: Gantt Chart Project Timeline

View full Gantt Chart here

Table 1: planned training metrics

Experiment ID	Reward type	Episodes	Average Episode reward	Success rate	Notes
1	N/A	N/A	TBC	N/A	TBC
2	N/A	N/A	TBC	N/A	TBC
3	N/A	N/A	TBC	N/A	TBC

Table 2: planned questionnaire results

Participant ID	Condition	Q1-Q7 average	Overall Rating	Comment
1	N/A	N/A	TBC	TBC
2	N/A	N/A	TBC	TBC
3	N/A	N/A	TBC	TBC

5.0 Feasibility, Risks, and Ethical Aspects

5.1 Feasibility

The project is attainable with the allotted time and resources. The software required includes Unity and Visual Studio, both of which are readily available and align with the project's technical requirements. Reinforcement learning will be implemented through C# scripts in Unity, supported by the ML-Agents framework to simulate learning behaviours and evaluate results (Juliani, et al., 2018). This setup allows for

efficient testing and experimentation without additional costs or third-party software. The project scope remains realistic since the focus is on improving AI behaviour within a controlled game environment, not creating a large-scale commercial product. Prior experience with Unity and programming will help provide the skills to develop a working prototype and effectively assess the reinforcement learning model. The Gantt chart outlines distinct and achievable phases, including literature review, model development, testing , and documentation (Figure 4). The prototype's modular structure also ensures smoother development and debugging, supporting the project's feasibility within the academic semester.

5.2 Risk Analysis and Mitigation

Table 3: Risk Assessment and Mitigation

Potential Risk	Likelihood	Impact	Mitigation Strategy
Difficulty integrating reinforcement learning with Unity game logic	Medium	High	Follow Unity ML-Agents documentation closely, test integration on smaller prototypes first before scaling up.
Limited understanding of reinforcement learning algorithms	Medium	Medium	Dedicate early project weeks to studying and experimenting with basic RL models (Q-learning, PPO, etc.) and seek supervisor guidance.
Performance issues when training AI agents in Unity	Medium	Medium	Optimise training environments using lower resolution, limited agents, and simplified physics to reduce computational load.
Time constraints during final testing and analysis phase	High	Medium	Maintain a consistent schedule using Gantt chart, prioritise a minimum viable prototype early to ensure core functionality is complete.
Data loss or project corruption	Low	High	Regularly back up project files to cloud storage and local drives after each major change.

5.3 Ethical Considerations

The project is not going to directly involve human participants or sensitive data, making the project low risk from the perspective of ethical engagement, however, ethical stands are still applied to responsibly develop and assess the project. All the datasets or an AI training done will either come from the public domain, or will be self-generated within the Unity environment, in order to avoid any privacy issues or concerns with data protection.

The reinforcement learning agents will operate solely inside the of the game world, ensuring no real-world consequences or harm. The project will also maintain academic integrity by properly referencing all research sources and ensuring transparency in documenting the design and development process. Any assets or libraries used will follow open-source licensing rules. The project's overall goal is to improve AI's adaptability in gameplay, which will allow for innovation in the interactive and creative technology, without posing any ethical or social risks.

Limitations

The expected limitations include training time, potential bad generalisation to unseen levels, and designer control limitations. These will be addressed in the final report and mitigated by a small, controlled experiment and clear documentation of training parameters.

Future work

Potential extensions include transfer learning to new levels, multi agent interactions, and a designer facing tool for tuning reward parameters instead of retraining.

References

Anon, 2018. *Skilled Experience Catalogue: A Skill-Balancing Mechanism for Non-Player Characters using Reinforcement Learning*. s.l., s.n.

Bontrager, P., Khalifa, A., Mendes, A. & Togelius, J., 2018. *Automatic playtesting for game parameter tuning via deep reinforcement learning,* s.l.: s.n.

Datatonic, T., 2022. *Reinforcement Learning: How to Train an RL Agent from Scratch*. [Online]
Available at: https://blog.montrealanalytics.com/reinforcement-learning-how-to-train-an-rl-agent-from-scratch-96af39a3287
[Accessed 30 October 2025].

Hester, T., Vecerik, M., Pietquin, O. & al., e., 2018. *Deep Q-Learning from Demonstrations,* s.l.: s.n.

Isbister, K. & Nass, C., 2000. *How People Talk to Computers, Robots, and Other Media*. San Francisco: Morgan Kaufmann.

Jaderberg, M., Czarnecki, W., Dunning, I. & al., e., 2019. Human-level performance in first-person multiplayer games with population-based deep reinforcement learning. *Science,* 364(6443), p. aau6249.

Juliani, A., Berges, V., Vckay, E. & al., e., 2018. *Unity ML-Agents: A toolkit for reinforcement learning and imitation learning in games.* [Online]
Available at: https://github.com/Unity-Technologies/ml-agents
[Accessed 27 October 2025].

Khan, A., 2014. *Pathfinding example: computed path from waypoint C to A.* [Online]
Available at: https://www.researchgate.net/figure/Pathfinding-example-computed-path-from-waypoint-C-to-A_fig5_262729027
[Accessed 30 October 2025].

Lample, G. & Chaplot, D., 2017. *Playing FPS games with deep reinforcement learning.* s.l., s.n.

MLVU, 2022. *Lecture 13: reinforement learning.* [Online]
Available at: https://mlvu.github.io/lecture13/
[Accessed 29 October 2025].

Mnih, V. et al., 2015. Human-level control through deep reinforcement learning. *Nature,* 518(7540), p. 529–533.

Pan, S. & Yang, Q., 2010. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering,* 22(10), p. 1345–1359.

Senanayake, C., 2025. *DYNAMIC NPC AI USING REINFORCEMENT LEARNING FOR AN ENHANCED GAMING EXPERIENCE.* s.l., s.n.

Silver, D., Schrittwieser, J., Simonyan, K. & al., e., 2017. Mastering the game of Go without human knowledge. *Nature,* Volume 550, p. 354–359.

Sutton, R. & Barto, A., 2018. *Reinforcement Learning: An Introduction.* 2nd ed. Cambridge, MA: MIT Press.

Tang, Z., 2023. *Reinforcement Learning in Digital Games: An Exploration of AI in Gaming.* s.l., s.n.

Van Hasselt, H., Guez, A. & Silver, D., 2016. *Deep reinforcement learning with double Q-learning.* s.l., s.n., p. 2094–2100.

Zook, A., Fruchter, E. & Riedl, M., 2019. *Automatic Playtesting for Game Parameter Tuning via Active Learning,* s.l.: s.n.

Appendices

Appendix A – Planned participant questionnaire (Likert 1-5)

Instructions: For each item choose between 1 (strongly disagree) to 5 (strongly agree).

Q1. The AI behaved responsively.

Q2. The AI felt lifelike.

Q3. The AI adapted to my actions during the session.

Q4. Playing against this AI was enjoyable.

Q5. The AI felt predictable.

Q6. I preferred the RL agent over the scripted baseline.

Q7. The AI punished/rewarded actions in a way that made sense.

Comments: Please add any comments about the AI behaviour or enjoyment.

Notes: This questionnaire is planned for the testing phase only, no participant responses have been collected for this interim report.

Appendix B – planned hyperparameters and experiment

Table 4: planned hyperparameters and experiments

Experiment ID	Algorithm	Learning rate	Gamma	Batch size	Buffer	Max steps	notes
E1 (Baseline)	Scripted	N/A	N/A	N/A	N/A	N/A	Scripted FSM/BT baseline
E2	PPO	3e-4	0.99	1024	2048	5000 ep	Default PPO
E3	DQN (test)	1e-4	0.99	32	10000	5000 ep	For discrete tasks (Hester, et al., 2018)

Appendix C – Data logging

Table 5: training data logging template

Run ID	Experiment ID	Seed	Start - End time	Episodes	Avg. episodic reward	Success rate	Collision	Note
0	0	0	0-0	N/A	N/A	N/A	N/A	N/A